Adversarisk kontrol af værnene

7/11 2026

Værnene var gjort grønne ved at bygge et undtagelsesregister og en forbrugsmekanisme for planlagte scope-ændringer. Det rejser det rigtige spørgsmål: **bider de stadig?** Eller er registret blevet et smuthul, hvor en commit kan slippe igennem med hvad som helst, blot dens SHA står på listen?

Alle forsøg nedenfor er kørt i en scratch-worktree på samme commit som `main` (`0165d79`), med `git commit --no-verify` hvor commit-msg-hooken ellers ville stoppe forsøget. Intet forsøgsarbejde er pushet.

1. Resultat — tre huller, alle nu lukket

#	Hul	Målt før rettelsen	Efter rettelsen
H1	Registret lempede hele regelsættet for en registreret SHA, ikke kun den regel der var efterprøvet	En commit uden Modul: og med tal der ikke findes i historikken blev godkendt, så snart dens SHA stod i registeret ([OK] … undtaget: verificeret register — test)	Rød: mangler linjen 'Modul:'. Kun den nævnte regel lempes
H2	scope_change_pending blev fjernet for alle projekter, når blot én række blev skrevet	Flag sat på site forsvandt, mens det var plugin der fik rækken — flaget var brændt uden at gælde noget	Flaget bliver stående; kun den række det gælder forbruger det
H3	scope_change: true krævede ingen begrundelse, og flaget accepterede true/tom streng	Et fald på 25,3 pp slap igennem med module: "daglig måling" og ingen årsag	Rød: scope_change dækker et fald på 25.3 pp, men "module" er 'daglig måling' — skriv hvad der blev ændret

2. Forsøgene, med de faktiske svar

Commit-værnet

Forsøg	Besked	Svar
A1	Samme fejl som 1c15f939 (Grundlag citerer modul-deltaet), ny SHA	RØD. Grundlag matcher ikke historik-rækken: 'P1.18 0,5 d af 29,65 d (P1 10,7 → 11,2 af 15,2 d)' matcher ikke rækkens '22,75 af 29,65 d (…)
A2	c747a47as tal (91,0 → 96,2 %), ny SHA	RØD. Før/Efter/Δ (91,0 % → 96,2 %, Δ +5,2) matcher ingen historik-række i projektstatus.json — ret tallene i JSON, kør --update og brug rækkens tal
A3	Registrerede tekst uden Modul:, ny SHA	RØD. mangler linjen 'Modul:' + Grundlag-fejlen
A4	A3's commit, men SHA'en skrevet i registeret med relax: ("history",)	RØD efter rettelsen (før: GRØN — det var H1). history-lempelsen dækker ikke formkravene
A4b	Samme registrering, men beskeden har «netop» den verificerede fejl	GRØN: register: test-post 2 (lempet: Før/Efter/Δ mod en historik-række)
A5	De to rigtige registrerede commits	GRØN med hver sin konkrete grund, fx 1c15f939 register: række 96 (76,3 → 76,7 %, 22,75 af 29,65 d): Før/Efter/Δ er rækkens egne tal, men Grundlag citerede modul-deltaet …
A6	Opslag på 1c15f9 (6 tegn), 1c15f93 (7), fuld SHA, deadbeef, tom streng	6 tegn → ingen træfning; 7 tegn og derover → træfning; ukendt/tom → ingen. Registret har 2 poster

Status-drift (projektstatus)

Forsøg	Svar
B1: s3.done hævet 0,2 d i JSON uden --update	RØD. docs/projektstatus.md stemmer ikke med projektstatus.json — blokken 'daglig' er ændret uden om scriptet (drift) — kør --update
B2: en historik-rækkes after rettet i hånden	RØD. history[115].delta (-0.7) passer ikke med after - before (22.0)
B3: sidste historik-række slettet	RØD. blokken 'log' er ændret uden om scriptet (drift)

Scope-ændringer

Forsøg	Svar
C1: flag på site, kun plugin får en række	Grøn kørsel, men flaget står stadig (før rettelsen: flaget forsvandt — H2)
C2: håndskrevet række med scope_change: true, generisk modul og 25,3 pp fald	RØD efter rettelsen (før: intet svar — H3)
C3: fald i plugin mens flaget sidder på site	RØD. history[116]: procenten falder (96.3 → 89.6) uden "scope_change": true — fald kræver dokumenteret årsag
C4: flag som true, og som "kort"	RØD. projekt 'site'.scope_change_pending skal være en sætning der forklarer scope-ændringen (fik True / 'kort')
C5: rigtig scope-ændring med begrundelse	GRØN. Rækken får module: "scope-ændring i test: 0,5 d lagt til S0", 65,3 → 62,3, scope_change: true, flaget forbruges, og --check er grøn bagefter

3. Hvad rettelsen ændrede

- **tools/commit_check.py:** `VERIFIED_LEGACY` er nu `SHA -> {relax: (regel, ...), reason}`. `relax` kan kun indeholde `basis` (Grundlag mod rækkens basis) eller `history` (Før/Efter/Δ mod en række). `check_message`/`check_history` tager imod lempelsen og springer kun den ene kontrol over; form, `? = Efter ? Før` og regnestykket valideres altid. Seks nye register-fixtures måler netop det.
- **projektstatus.py:** `consume_scope_flags()` fjerner kun flaget for det projekt der fik en række; `scope_change_pending` skal være en sætning på mindst 8 tegn; en `scope_change`-række med et fald på **≥ 0,5 pp** skal navngive ændringen i `module` (fx `scope-korrektion S2.6`) — under tærsklen regnes faldet som afrundingsstøj. Fem nye fixtures måler det.

Tærsklen på 0,5 pp er valgt, fordi en eksisterende række på `main` (96,4 → 96,3) er afrundingsstøj; den må ikke gøre historikken ugyldig bagefter. Alt derover kræver en navngivet årsag.

3b. Den retroaktive fejning: historikken er ikke grøn, og det er meningen

Et nærliggende spørgsmål er, om «hele» historikken består, hvis man fejer den igen. Det gør den ikke, og det skal stå her, så ingen forsøger at "rette" det med registeret:

Fejning	Resultat
Rangen siden sidste push (det CI faktisk tjekker: before..after)	52 af 52 grønne
De 40 nyeste commits på main	40 af 40 grønne
Hele historikken (422 commits)	129 røde — heraf 124 fra 3/11 og frem

De 124 er skrevet **før** krydstjekket mod historikken og formkravene blev skærpet (281 manglende felter som `?:`, 37 `Grundlag`-ordlyder og 18 `Før`-felter med fritekst). De er ikke registreret og skal ikke registreres: registret findes til commits der **kan** forklares mod en række, ikke til at gøre gammel historik grøn. CI fejer kun den range der blev pushet, så de gamle commits mødes aldrig af værnene

igen — og det er netop grunden til at de ikke blokerer noget i dag.

4. Sådan gentages kontrollen

```
git worktree add --detach .tools/adv HEAD      # scratch, aldrig main
# ... kør forsøgene med `git commit --no-verify` og `--commit`/`--range`
git worktree remove .tools/adv
```

Tre ting er værd at huske:

- **Hooken bider også.** Uden `--no-verify` blev forsøgs-committene afvist af `commit-msg`-hooken, før de overhovedet blev til commits — det var i sig selv et bevis, men det gør også, at en `--commit HEAD`-kontrol kan ende med at måle den «forrige» commit, hvis man ikke tjekker exitkoden.
- **Drift testes med --check, ikke --update.** `--update` skriver historikken og skjuler dermed netop den drift man vil fange.
- **Et grønt --update beviser ikke, at flaget blev brugt rigtigt.** Læs rækken bagefter: hvilket projekt fik den, og står der en begrundelse i `module`.